Design and Implementation of a Web System for Searching and Managing Academic Courses

Brayan Alejandro Riveros Rodríguez
Dept. of Computer Engineering
Universidad Distrital Francisco José de Caldas
Email: bariverosr@udistrital.edu.co

Carlos Andres Pescador Castro
Dept. of Computer Engineering
Universidad Distrital Francisco José de Caldas
Email: capescadorc@udistrital.edu.co

Cristian Santiago Ríos Vásquez
Dept. of Computer Engineering
Universidad Distrital Francisco José de Caldas
Email: csriosv@udistrital.edu.co

***Abstract*—Efficient access to academic information has become a challenge for institutions and users seeking courses that meet their needs. In this context, we present the development of a web application called Course Search, which aims to facilitate course consultation and administration through an intuitive interface and modular design.**

The proposed solution integrates a search service capable of filtering and classifying courses according to defined criteria, along with an administration module exclusively for the administrator user, who manages the registered courses and teachers. The system was designed following service-oriented architecture principles, using UML and activity diagrams that support the clarity of the design and the correct structuring of the system.

***Index Terms*—course search engine, web systems, academic management, service-oriented modular architecture, process optimization.**

I. Introduction

In the context of today's higher education, internal management of academic information represents a constant challenge for institutions. In many cases, students and professors at the same university face difficulties when trying to locate available courses or identify who teaches them, because the data is scattered across different platforms or institutional documents. This lack of centralization limits efficiency in consultation and hinders academic planning.

In this scenario, there is a need to develop a specialized search system for educational institutions that allows quick and organized access to academic offerings. This system aims to optimize interaction between users and the university's internal information, improving transparency, accessibility, and the search experience.

Previous research on academic information management and search systems has focused on improving access to course-related data within educational institutions. For example, the University of Tartu developed an online study information system that allows students and staff to browse, filter, and retrieve detailed course information through a centralized institutional platform [1]. This system exemplifies how universities can use digital infrastructures to centralize course data and make it easily accessible to their academic communities. Similarly, commercial solutions such as the Terminalfour Course Search module provide customizable search engines for higher education institutions, enabling students to explore available programs and subjects based on filters such as academic area or level of study [2].

While both initiatives enhance discoverability and transparency of academic offerings, they remain oriented toward course visibility and student decision-making rather than modeling the structural relationship between courses and the professors who teach them. Furthermore, most existing solutions are designed to serve multiple departments or institutions through generalized catalog structures. In contrast, the present work focuses on a single academic institution and emphasizes the bidirectional association between courses and instructors, providing a more specific and administratively useful search system for institutional management and academic organization.

The work presented below is organized as follows: Section II describes the methodology and architecture used in developing the system; Section III presents the results obtained and their analysis; Section IV presents the conclusions and possible lines of future work.

II. Methods and Materials

The development of the proposed system is based on a layered architecture, designed to ensure a clear separation of responsibilities, promote maintainability, and facilitate the future scalability of the project. This structure allows the system components to be organized logically, so that each layer fulfills a specific purpose within the information flow, from user interaction to persistent data management.

The presentation layer corresponds to the environment where users interact, offering an intuitive and functional interface that allows them to search for courses and teachers within an educational institution. This layer communicates with the business logic through controlled requests, preventing

direct access to data and preserving the integrity of operations. The interface has been designed with the aim of offering a fluid experience, prioritizing accessibility and clarity in navigation.

The business logic layer concentrates the main rules of the system and defines the processes that enable coordination of operations between different entities. This layer includes components such as AuthService, which manages administrator authentication; SearchService, which processes course and professor searches; and ProfessorCourseManager, which maintains the relationship between teachers and subjects. These classes act as mediators between the presentation and the data, ensuring that each user request is validated and processed correctly before accessing the stored information.

The data layer is responsible for the persistence and retrieval of information. It consists of the Course and Professor entities, which represent the fundamental objects of the academic domain. The relationship between the two entities is managed through associations controlled by the business layer, ensuring the consistency of registration, search, and update operations. This design facilitates the future extension of the model, allowing new attributes or entities to be included without compromising the overall stability of the system.

In terms of design, the class diagram illustrates the structural organization of the system and the interaction between the main components. Layered architecture promotes loose coupling and high cohesion, two essential principles for achieving modular and maintainable software. In this way, each layer can evolve independently, allowing, for example, the user interface to be replaced or improved without altering the business rules or data structure.

In addition, the model was designed with clarity and reproducibility in mind. All design decisions were made seeking a balance between simplicity and functionality, so that the implementation can be replicated or extended by other developers under the same conceptual premises.

Finally, the restriction to a single educational institution and the lack of integration with external systems are recognized as initial limitations, aspects that may be addressed in later versions of the project.

The system architecture follows a microservices approach, comprising three main components that communicate through RESTful APIs. The frontend was developed using React, providing a responsive and dynamic user interface that adapts to different devices and screen sizes. This choice enables a modern single-page application experience, where users can interact with the system without constant page reloads.

The backend infrastructure is divided into two specialized services. The first service, built with Java Spring Boot, is exclusively dedicated to authentication and authorization. This service implements JWT (JSON Web Token) based authentication, ensuring secure access control for administrative operations. It connects to a MySQL database that stores administrator credentials and manages session tokens. The second service, developed with Python FastAPI, handles all business logic operations including course management, professor administration, and search functionality. This service communicates with a PostgreSQL database that maintains the complete academic catalog and the relationships between courses and professors.

Figure 1 presents the class diagram of the system, illustrating the organization across four distinct layers. The Controllers layer manages HTTP requests and routes them to appropriate services. Key controllers include ProfessorController and CourseController for entity management, AssignmentController for handling professor-course relationships, SearchController for query processing, and AuthController for authentication operations. The Business Services layer contains the core logic through components such as ProfessorCourseManager, which maintains the many-to-many relationship between professors and courses, SearchService for implementing search algorithms, and AuthService for credential validation.

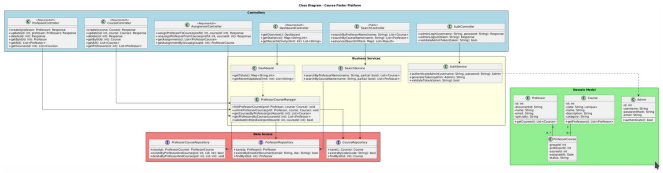

Fig. 1. Class diagram showing the layered architecture of the Course Finder system, including Controllers, Business Services, Data Access, and Domain Model layers.

The Data Access layer implements the repository pattern, providing abstraction for database operations through ProfessorRepository, CourseRepository, and ProfessorCourseRepository interfaces. These repositories handle CRUD operations and specialized queries. Finally, the Domain Model layer defines the core entities: Professor (with attributes id, username, name, and specialty), Course (with id, code, name, credits, and category), and ProfessorCourse (representing the many-to-many relationship with professorId, courseId, groupId, assignedAt, and status attributes).

The development process incorporated continuous integration and continuous deployment (CI/CD) practices using Docker containerization and GitHub Actions workflows. Quality assurance was ensured through multiple testing levels: unit tests using JUnit for Java components and pytest for Python services, acceptance tests implemented with Cucumber

to validate user stories, and stress tests conducted with JMeter to evaluate system performance under load. Maven was used as the build automation tool for the Java service, managing dependencies and compilation processes.

The SearchService supports multiple search criteria, including professor name-based queries (returning all courses taught by matching professors), course name searches (with partial matching capabilities), and advanced filtering options that combine multiple parameters. All search operations return results in a standardized format that includes complete course information and associated professor details.

Administrative users have full CRUD capabilities over both professors and courses through authenticated endpoints. The create operations validate unique constraints (such as email uniqueness for professors and code uniqueness for courses) before persisting entities. Update operations support both full replacement and partial modifications. Delete operations check for existing relationships before removal, preventing data inconsistencies. Additionally, administrators can assign and unassign professors to courses, managing the academic schedule through the AssignmentController.

This architectural design provides several advantages. The separation between authentication and business logic services allows independent scaling based on demand. The use of different database systems (MySQL for authentication, PostgreSQL for business data) optimizes each component for its specific workload. The layered structure within each service promotes code reusability and simplifies maintenance. However, the system currently operates within a single institution context and does not integrate with external academic platforms, aspects that could be addressed in future iterations.

III. Results & Discussion

The functional testing stages (unit and end-to-end) showed that all core workflows behaved correctly, confirming the internal consistency of both backends and the correctness of the implemented user stories. Although these results validated product quality, the most relevant findings came from the performance evaluation stage, which exposed the system's behavior under high concurrency and realistic enrollment-period conditions.

To assess scalability, a stress-testing scenario was executed using Apache JMeter, simulating 15,000 concurrent users performing search operations on the business logic backend. This scenario reflects the peak load expected during university registration periods, where thousands of students simultaneously query courses, professors, and assignments. The search endpoint showed an average response time of approximately 23 seconds (Figure 2), becoming the main bottleneck of the system under heavy load. Minimum and maximum response times displayed high variance as the system approached saturation (Figure 3). Throughput peaked at around 240 requests per second for the search endpoint, while all other operations

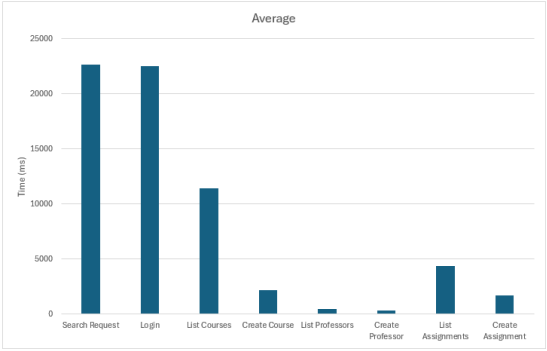

Fig. 2. Average response times under peak load.

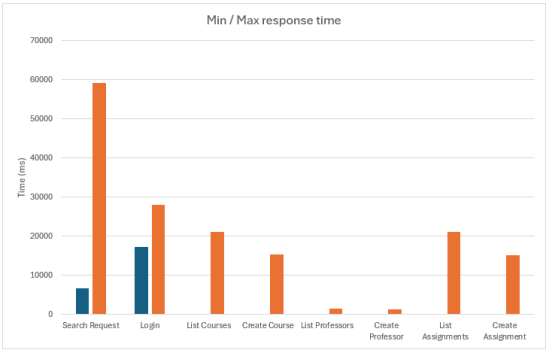

Fig. 3. Minimum and maximum response times recorded during the stress test.

remained below 1 request per second due to their lower expected frequency (Figure 4).

These results must be interpreted considering the hardware used during testing: both backends were deployed on a single local machine equipped with 8 GB RAM and a 4-core CPU. Under these constraints, resource contention significantly impacted overall system responsiveness, especially during concurrent search operations. The stress tests therefore do not indicate architectural failure but rather highlight the limitations of the execution environment. The system remains functionally correct and stable, but its ability to support real-world academic load depends on improved infrastructure, distribution

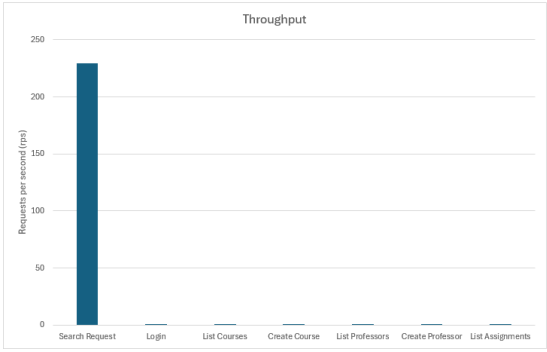

Fig. 4. Throughput measured during high-concurrency execution.

of services across separate hosts, and further optimization of database queries and caching strategies.

IV. Conclusions

The project successfully achieved its objectives by delivering a reliable web-based system that centralizes course and instructor information, provides efficient bidirectional search, and supports key administrative operations. The problem context, stakeholders, and terminology were synthesized into a consistent baseline that guided the architectural decisions and the implementation roadmap. All unit and end-to-end tests passed, confirming the correctness of authentication, course management, and search functionalities, and validating that the system meets stakeholder expectations and aligns with the planned development path.

However, the stress tests revealed clear performance limitations, particularly for the search endpoint under high concurrency, largely due to the restricted hardware used during evaluation. These findings emphasize the need for stronger deployment infrastructure and performance optimizations before the system can support real peak academic load. Future work should focus on improving scalability—through better hosting, query optimization, and caching—while also exploring functional extensions such as enrollment tools, dashboards, and enhanced scheduling capabilities.

References

[1] U. of Tartu. (2015) Access to course information: The online study information system at the university of tartu. Erasmus+ European Commission. Accessed: 2025-10-20. [Online]. Available: https://erasmus-plus.ec.europa.eu/eche/access-to-course-information-the-online-study-information-system-at-the-university-of-tartu

[2] Terminalfour. (2023) Course search solutions for higher education. Terminalfour. Accessed: 2025-10-20. [Online]. Available: https://www.terminalfour.com/solutions/course-search/